

Practical 1

Title: Linear regression by using Deep Neural network: Implement Boston housing price Prediction problem by Linear regression using Deep Neural network. Use Boston House price prediction dataset.

Aim:

To implement **Boston House Price Prediction** using **Linear Regression with a Deep Neural Network (DNN)** and evaluate the model performance using suitable metrics.

1. Introduction

Linear Regression is a supervised machine learning algorithm used to predict a **continuous output value** based on one or more input features.

In this practical, we solve the **Boston Housing Price Prediction problem**, where the goal is to predict the **median value of houses (price)** using different features such as crime rate, number of rooms, accessibility to highways, etc.

Instead of using traditional linear regression formula directly, we implement linear regression using a **Deep Neural Network**.

2. Boston Housing Dataset

The **Boston Housing dataset** is a famous regression dataset that contains information collected by the U.S Census Service about housing in Boston.

Dataset contains

- **506 samples (rows)**
- **13 input features (columns)**
- **1 output target value (house price)**

Target

- **MEDV** → Median value of owner-occupied homes in \$1000s

3. Features in Dataset

Some important features are:

Feature	Description
CRIM	Crime rate per capita
RM	Average number of rooms per dwelling
AGE	Proportion of old houses
TAX	Property tax rate
LSTAT	% lower status population
PTRATIO	Pupil-teacher ratio
INDUS	Proportion of non-retail business area

These features affect the house price.

4. Linear Regression Concept

In traditional linear regression, output is predicted using:

$$y=w_1x_1+w_2x_2+...+w_nx_n+b$$

Where:

- $x_1, x_2, \dots, x_n$ are input features
- $w_1, w_2, \dots, w_n$ are weights
- b is bias
- y is predicted house price

The objective is to find the best values of **weights and bias** such that predicted value is close to actual value.

5. Linear Regression using Deep Neural Network

A **Deep Neural Network** can also implement linear regression by using:

Single Dense Layer (Linear Output)

If we use only one neuron in output layer with **linear activation**, then it acts as linear regression.

$$y=Wx+ b$$

In neural networks:

- weights are learned during training
- bias is also learned
- training happens using optimization algorithms like Gradient Descent / Adam

6. Deep Neural Network Architecture

For Boston housing regression, we can use:

- Input Layer: 13 neurons (for 13 features)
- Hidden Layers: 1 or more Dense layers with ReLU activation
- Output Layer: 1 neuron with Linear activation

Example architecture:

- Dense(64, activation='relu')
- Dense(32, activation='relu')
- Dense(1, activation='linear')

7. Activation Function

ReLU Activation

Used in hidden layers:

$$\text{ReLU}(x) = \max(0, x)$$

It helps the network learn non-linear relationships.

Linear Activation

Used in output layer for regression:

$$f(x) = x$$

Because output is continuous, we use linear activation.

8. Loss Function

For regression problems, we use **Mean Squared Error (MSE)**:

$$\text{MSE} = \frac{1}{n} \sum (y_{\text{true}} - y_{\text{pred}})^2$$

MSE measures average squared difference between actual and predicted values.

9. Optimization Algorithm

To reduce loss, the model uses **Gradient Descent** based optimizers like:

Adam Optimizer

Adam is widely used because it provides faster convergence.

It updates weights using gradient information to minimize loss.

10. Model Training Process

The training steps are:

1. Load dataset
2. Split into training and testing data
3. Normalize/Standardize features
4. Create DNN model
5. Compile model with loss function and optimizer
6. Train model using epochs
7. Evaluate model performance

11. Backpropagation

- Backpropagation is a process used to compute gradients of loss with respect to weights.

Steps:

- Forward propagation → prediction
- Calculate loss
- Backward propagation → calculate gradients
- Update weights
- This process repeats for many epochs.

12. Optimizer Used: Adam

- Instead of simple gradient descent, we use **Adam optimizer**.

Adam stands for: **Adaptive Moment Estimation**

- Advantages:
 1. faster convergence

2. automatically adjusts learning rate
3. better for deep learning regression tasks

13. Data Preprocessing Requirement

In Boston dataset, feature ranges differ widely.

Example:

1. CRIM values can be small
2. TAX values can be large
3. If we do not scale data:
4. large features dominate learning
5. model becomes unstable
6. slow convergence

14. Normalization / Standardization

Since dataset features have different ranges, preprocessing is necessary.

Standardization is done as:

$$z = \frac{x - \mu}{\sigma}$$

Where:

- μ = mean
- σ = standard deviation

This improves training speed and accuracy.

Benefits:

1. faster training
2. better gradient descent performance
3. improved accuracy

15. Train-Test Split

Dataset is divided into:

- **Training Data**

Used to train model (usually 80%)

- **Testing Data**

Used to evaluate model performance (usually 20%)

This ensures model is tested on unseen data.

16. Epoch and Batch Size

- **Epoch**

One complete pass through entire training dataset.

Example:

If epochs = 100, model learns 100 times from training data.

- **Batch Size**

Number of samples processed before updating weights.

Example:

batch size = 32 means weights update after every 32 samples.

17. Performance Metrics

To evaluate regression model:

1. Mean Absolute Error (MAE)

$$MAE = \frac{1}{n} \sum |y_{true} - y_{pred}|$$

2. Mean Squared Error (MSE)

$$MSE = \frac{1}{n} \sum (y_{true} - y_{pred})^2$$

3. Root Mean Squared Error (RMSE)

$$RMSE = \sqrt{MSE}$$

RMSE gives error in same unit as output (price).

Conclusion

In this practical, Boston house price prediction is performed using a **Deep Neural Network model** that behaves like linear regression.

The model learns weights using backpropagation and optimizer to minimize prediction error.

Thus, DNN provides an efficient way to solve regression problems like house price prediction.